YOUR PERSONALIZED STUDY DECK

AI engineering, explained through your work

72 flashcards + 4 coding drills

Production GenAI / Python / AI engineering / MCP

What changed from the earlier answers

Your MFlash and NoteEchoes briefings now supply the exact model roles, configured training settings, retrieval parameters, evaluation denominators, and recovery mechanisms that were previously left for you to check.

The short version of your story

Bank of America demonstrates production workflow delivery. NoteEchoes demonstrates specialized local AI and reliable capture. MFlash demonstrates the boundary between model proposals and controlled execution.

How to study a card

Cover the answer. Read the question and speak for 30-60 seconds. Check the answer, then repeat the memory cue without looking. Use the follow-up line for deeper practice.

Evidence matters

Project details come from the two supplied technical briefings; this guide did not inspect their repositories or rerun their tests. Saved results, configured settings, proposed improvements, and operational evidence are kept distinct.

STUDY METHOD

A simple five-session practice plan

Session 1 | Your story and the must-haves

Study cards 01-12 and 61-68. Practice your introduction, production example, and MCP explanation. Say exactly which project supports each claim.

Session 2 | NoteEchoes

Study cards 13-32. Draw capture -> save -> enhance -> validate. Explain the 96.3% score and its template-overlap limitation without reading.

Session 3 | MFlash

Study cards 33-52. Draw proposal -> grant -> execution -> receipt. Practice the crash window, changed-source example, and ranking-versus-task-success distinction.

Session 4 | Engineering and Python

Study cards 53-60 and 69-72. Write the coding drills from a blank file. Talk through error policies, cancellation, idempotency, and authorization.

Session 5 | Mock interview

Answer 10 shuffled cards, then spend five minutes drawing an airline rebooking system. Review only the cards you missed. Repeat difficult cards tomorrow and again three days later.

Self-score every answer

0 = cannot explain. 1 = partly correct. 2 = clear and specific. 3 = clear, specific, and handles the follow-up. Prioritize weak cards rather than repeatedly rereading familiar ones.

FLASHCARDS

Cards 01-03

EASY 01 / CORE

Tell me about yourself.

I am an AI engineer with experience in production document extraction, Python services, and data pipelines. At Bank of America, my work includes fine-tuned extraction, validated JSON delivery, and MCP retrieval. My projects add two complementary examples: NoteEchoes for local model integration and reliable note capture, and MFlash for constrained action proposals and checked execution. This role combines those AI and software-engineering strengths.

REMEMBER: Production + Python + local AI + controlled actions.

FOLLOW-UP: Keep this to 60 seconds. Use Bank of America for production experience; the projects for technical depth.

EASY 02 / CORE

How do your projects match the four must-haves?

For production GenAI and MCP, my strongest resume example is Bank of America's extraction and document-retrieval work. For Python and AI engineering, MFlash adds typed execution, classifiers, ranking, and local retrieval. NoteEchoes adds Qwen adaptation, evaluation, mobile inference, and persistence safeguards. NoteEchoes itself is Flutter/Dart with Swift integrations, not a Python or Node.js application.

REMEMBER: Use the right project for each requirement.

FOLLOW-UP: Do not present MFlash or NoteEchoes as MCP implementations: neither briefing establishes that integration.

EASY 03 / CORE

What is the difference between an LLM and a classifier?

A classifier selects among labels. An LLM generates token sequences. MFlash's structured classifier selects action templates; it is a randomly initialized MLP, not a fine-tuned LLM. NoteEchoes uses a specialized pretrained Qwen3 model to produce structured action analysis. I choose a model according to the task's output space and operational constraints.

REMEMBER: MLP selects; Qwen generates.

FOLLOW-UP: MFlash also has an MPNet candidate ranker, which is a third, separate role.

FLASHCARDS

Cards 04-06

EASY 04 / CORE

Explain transformers in simple terms.

A transformer uses attention to combine relevant information from tokens, plus feed-forward layers to transform representations. In a causal language model, attention is masked so a position cannot use future tokens. NoteEchoes adapts a pretrained transformer; it does not pretrain a foundation model. MFlash's structured MLP is not a transformer.

REMEMBER: Attention connects context; adaptation specializes behavior.

FOLLOW-UP: If asked for the formula: softmax(QK-transpose / sqrt(d_k)) V.

EASY 05 / CORE

What are embeddings, and where did you use them?

Embeddings represent inputs as numeric vectors for similarity comparisons. NoteEchoes uses multilingual E5-small with 384-dimensional normalized embeddings. MFlash has retrieval profiles using MPNet or EmbeddingGemma, and a separate MPNet action-ranking model. I keep model identity with embeddings because vectors from different spaces should not be directly compared.

REMEMBER: Same space before similarity.

FOLLOW-UP: Do not mix the action ranker's learned projections with document retrieval embeddings.

EASY 06 / CORE

When would you use retrieval versus fine-tuning?

Retrieval finds relevant stored information at query time. Fine-tuning adapts behavior. NoteEchoes uses Qwen adaptation for structured intent/action analysis and retrieval for saved notes and documents. Its active conversation and PDF paths return cited excerpts rather than using the action model as a general document chatbot.

REMEMBER: Retrieval finds facts; tuning shapes behavior.

FOLLOW-UP: A grounded retrieval feature does not necessarily use generative answer synthesis.

FLASHCARDS

Cards 07-09

EASY 07 / CORE

What is an agent, and are your apps agents?

An agent uses model decisions to choose actions toward a goal. My projects combine deterministic workflows with model-assisted components. MFlash ordinary chat uses typed intents and deterministic proposals for supported routes; its workbench classifier and visual experiments are separate paths. NoteEchoes capture enhancement does not execute arbitrary model-selected tools.

REMEMBER: Describe the actual path, not a broad agent label.

FOLLOW-UP: Explain why a fixed workflow can be simpler and safer for a bounded task.

EASY 08 / CORE

What is MCP?

MCP is a protocol for connecting AI applications to tools and context. My resume example is enterprise document retrieval at Bank of America. The host is the application, its client communicates with a server, and the server exposes capabilities. MCP standardizes integration; application code still needs authorization, validation, and reliable execution.

REMEMBER: Host -> client -> server.

FOLLOW-UP: The supplied project briefings do not fill in the Bank of America MCP transport or server ownership.

MEDIUM 09 / CORE

How is MCP different from REST and tool calling?

REST exposes application services over HTTP. Model tool calling expresses a proposed invocation. MCP standardizes how compatible applications discover and interact with capabilities and context. An MCP server can wrap REST services. In all cases, the trusted execution layer must authorize and validate the operation; a model proposal is not permission.

REMEMBER: Interface, proposal, protocol: different layers.

FOLLOW-UP: Use MFlash as an example of an execution boundary, not as evidence that it uses MCP.

Cards 10-12

EASY 10 / CORE

What is the difference between schema validity and correctness?

Schema validity means the output follows its required structure. Correctness means the content and action are right for the request. NoteEchoes records schema passes on 1,080 synthetic action rows, while selected-field agreement is 1,040 out of 1,080. A perfectly structured response can choose the wrong mode or tool arguments.

REMEMBER: Valid shape does not mean correct meaning.

FOLLOW-UP: This is one of the most important distinctions in your interview.

MEDIUM 11 / CORE

How do you evaluate an AI system in layers?

I separate model output, policy decisions, execution, and user outcome. NoteEchoes selected-field matching evaluates structured outputs. MFlash candidate ranking evaluates selection from supplied actions. Neither establishes successful real-world execution. I would separately evaluate retrieval, authorization failures, side effects, recovery, latency, and business outcomes.

REMEMBER: Output -> policy -> execution -> outcome.

FOLLOW-UP: Use a different denominator and acceptance criterion for each layer.

MEDIUM 12 / CORE

Explain Python async versus threads and processes.

Async is useful for concurrent network or other cooperative I/O. Threads can keep a UI responsive or wrap blocking I/O. Processes can parallelize CPU-heavy Python work. MFlash runs controller work in a background thread; that alone does not prove CPU parallelism. In standard GIL-enabled CPython, Python bytecode threads generally do not run in parallel.

REMEMBER: I/O: async; blocking work: threads; CPU: consider processes.

FOLLOW-UP: Never describe MFlash's background worker as asyncio unless the specific path uses it.

FLASHCARDS

Cards 13-15

EASY 13 / NOTEECHOES

Explain NoteEchoes in one minute.

NoteEchoes is a Flutter/Dart iPhone app with native Swift integrations for voice capture, local model inference, OCR, and device features. It saves notes and checklists, imports documents, and retrieves information from a personal collection. Its Qwen action interpreter enhances an already-saved note. Fingerprint checks protect later edits, and retrieval returns cited excerpts. Its synthetic evaluation results are useful regression evidence, with important generalization limits.

REMEMBER: Capture -> save -> enhance -> validate -> retrieve.

FOLLOW-UP: Position it as local-first application engineering, not proven large-scale production adoption.

MEDIUM 14 / NOTEECHOES

Walk through a spoken checklist request.

Recorded speech first attempts WhisperKit transcription, with guarded Apple Speech fallback. The service sanitizes the transcript, applies deterministic parsing, creates the note, persists it, and schedules indexing. Optional Qwen enhancement then analyzes the original transcript. CoreV5 validation checks the JSON. Before applying changes, the service compares the note fingerprint with its earlier snapshot and discards stale enhancement.

REMEMBER: Save first; enhance second.

FOLLOW-UP: This trace is an implemented request path, not a measured historical user session.

MEDIUM 15 / NOTEECHOES

Which exact model and runtime does NoteEchoes use?

The main action interpreter is a specialized Qwen3-0.6B model deployed as an 8-bit MLX artifact. Dart calls a Flutter method channel, and native Swift performs local MLX inference. The configured artifact is pinned to revision b829d1d480c0bc0226326e36f009ae825af60f18. All recognition-language settings route to this combined action model in the inspected implementation.

REMEMBER: Qwen3-0.6B -> MLX 8-bit -> Swift bridge.

FOLLOW-UP: Memorize model size and runtime; the full revision is reference material, not an interview recital.

FLASHCARDS

Cards 16-18

MEDIUM 16 / NOTEECHOES

What were the QLoRA training settings?

The initial training configuration uses Qwen3-0.6B, 4-bit NF4 with double quantization, and rsLoRA with rank 32, alpha 64, and dropout 0.05. Targets include attention projections and gate/up/down projections. It uses completion-only causal loss, maximum length 1,024, learning rate 4e-5, batch 8, accumulation 2, and one epoch. These are configured settings, not a verified lineage for every distributed weight.

REMEMBER: 0.6B / 4-bit / rank 32 / alpha 64 / 4e-5.

FOLLOW-UP: The initial trainer saves the final adapter; it does not load the best model at the end.

MEDIUM 17 / NOTEECHOES

Why 4-bit training but 8-bit deployment?

They are separate stages. QLoRA reduces training memory by keeping the base quantized while training adapters. The app uses a separately prepared 8-bit MLX deployment artifact. Training precision does not force deployment precision. I can explain that tradeoff, but the briefing does not establish that 8-bit was empirically optimal on devices.

REMEMBER: Training memory and deployment format are separate decisions.

FOLLOW-UP: Do not invent latency, memory, or battery benchmarks.

HARD 18 / NOTEECHOES

What does completion-only loss mean?

The training code masks prompt labels with -100, so loss is computed on the target completion instead of teaching the model to reproduce the input prompt. The model learns the desired structured output conditioned on the request. This is supervised adaptation of a pretrained model. A separate GRPO program exists, but there is no established evidence that it produced the deployed weights.

REMEMBER: Mask the prompt; learn the completion.

FOLLOW-UP: Do not describe the deployed model as RLHF-trained on the available evidence.

FLASHCARDS

Cards 19-21

MEDIUM 19 / NOTEECHOES

How large was your training dataset?

There are distinct packages. The original synthetic manifest lists 30,000 training requests paired into 60,000 SFT records. The English correction corpus has 6,000 training rows: 5,400 action and 600 normalization examples. Validation, test, and challenge each have 600 rows. The correction counts and hashes were checked in the supplied review; the original manifest counts were not freshly recounted.

REMEMBER: Original package != correction package.

FOLLOW-UP: Do not combine counts from different releases or call synthetic rows real-user requests.

HARD 20 / NOTEECHOES

What exactly does the 96.3% score mean?

It is 1,040 out of 1,080 archived synthetic action outputs passing the stored operational gate and exactly matching seven selected fields: intent, mode, items, entities, draft, requires_confirmation, and the full proposed_tool. It excludes title, language, confidence, normalized text, and clarification wording. The supplied review rescored saved outputs; it did not rerun model inference.

REMEMBER: 1,040 / 1,080; seven fields; synthetic archive.

FOLLOW-UP: Call it selected action-field exact match, not overall production accuracy.

HARD 21 / NOTEECHOES

How do the other NoteEchoes metrics differ?

The archive contains 1,200 cases: 1,080 action and 120 normalization cases. Stored operational flags pass 1,200/1,200, while strict-check flags pass 1,097/1,200, or 91.417%. Stored schema flags pass all 1,080 action rows; schema validation was not rerun in the review. Normalization matches 120/120. These metrics test different conditions and use different denominators.

REMEMBER: Operational != selected fields != strict object match.

FOLLOW-UP: The operational gate does not require full argument, mode, item, entity, or draft equality.

FLASHCARDS

Cards 22-24

HARD 22 / NOTEECHOES

Was the test data independent of training?

Files and row hashes are separate, but that does not ensure semantic independence. The supplied review normalized synthetic identifiers and found all 540 test action targets and all 540 challenge action targets matched templates present in training. That weakens evidence of generalization. It is not proof that identical test rows were intentionally trained on. I would add independently authored, scenario-disjoint holdouts.

REMEMBER: Different rows can share the same template.

FOLLOW-UP: This is a proposed next evaluation, not a completed improvement.

HARD 23 / NOTEECHOES

How much better was the model than stock Qwen?

I do not have a completed matched baseline comparison in the evidence provided. The 96.3% result is an absolute score on a specific synthetic evaluation, not an improvement over stock Qwen. I would compare stock and adapted models with the same prompts, decoding settings, runtime, and independently authored holdout, including per-language and failure slices.

REMEMBER: No baseline -> no improvement percentage.

FOLLOW-UP: Do not turn a fine-tuned model's score into a gain over its base model.

MEDIUM 24 / NOTEECHOES

Which errors remained in the saved outputs?

The supplied rescore found 22 mode differences, 14 complete tool-object differences, and 6 draft differences. Those categories overlap, so I would not add them to count unique failures. There were 40 failed selected-field cases overall. I would inspect mode selection and full tool arguments separately rather than focusing only on schema validity.

REMEMBER: 40 failed cases; error categories overlap.

FOLLOW-UP: No selected item/entity differences were found in that archive; that is not a universal guarantee.

FLASHCARDS

Cards 25-27

MEDIUM 25 / NOTEECHOES

Explain NoteEchoes retrieval precisely.

The hybrid note path combines SQLite FTS5/BM25 with optional multilingual E5-small semantic matches. It takes up to 15 lexical and 15 semantic matches and combines ranks with 1/(60+rank). One-hop relationships can add 0.005, and the default output is six candidates. Stored vectors are scanned directly. The inspected path has no learned reranker.

REMEMBER: 15 + 15 -> reciprocal rank fusion -> 6.

FOLLOW-UP: These are note-retrieval settings, not a claim that every document path uses identical settings.

MEDIUM 26 / NOTEECHOES

How are PDFs parsed and chunked?

The processor uses selectable PDF text and positional information to approximate reading order. Sparse-text pages attempt native Apple Vision OCR. Target chunks are 1,800 characters with 220-character overlap, preferring paragraph, line, or sentence boundaries. Chunks retain page and source-order information. These are character counts, not token counts.

REMEMBER: Text first -> OCR fallback -> 1,800 / 220.

FOLLOW-UP: Layout annotations do not establish full diagram or table understanding.

MEDIUM 27 / NOTEECHOES

How does the knowledge graph work?

It has semantic note relationships and connected-component topic grouping. The visible graph displays topics and their member notes, with up to 60 note nodes. That is useful for navigation and organization. I would describe it as a note/topic graph, not an entity-predicate reasoning engine or a graph database.

REMEMBER: Related notes form topics; graph shows membership.

FOLLOW-UP: The review identifies asymmetric relationship expansion as a potential improvement, not a completed fix.

FLASHCARDS

Cards 28-30

HARD 28 / NOTEECHOES

What if a user edits a note during inference?

The initial note is already saved. Before enhancement, the service records a fingerprint. When the model returns, it compares that fingerprint with the current note. If the note changed or was deleted, the enhancement is discarded. That protects user edits from late model output. A regression test was inspected in the briefing, but it was not run during that review.

REMEMBER: Snapshot -> generate -> compare -> apply or discard.

FOLLOW-UP: A Dart timeout alone does not prove native inference stopped.

HARD 29 / NOTEECHOES

How do you prevent deleted notes from returning after recovery?

Primary storage uses SQLite, with recovery snapshots and deletion tombstones. Recovery chooses a complete valid snapshot rather than unioning old snapshots, combines deletion IDs, and skips tombstoned notes. This addresses stale backups containing deleted records. Snapshot mirroring can finish after note saving, so I would not claim every mirror is synchronous or power-loss-proof.

REMEMBER: Choose one snapshot; respect tombstones.

FOLLOW-UP: SQLite uses WAL with synchronous=NORMAL; no device power-loss experiment is established.

MEDIUM 30 / NOTEECHOES

Does the model execute reminders or arbitrary tools?

The capture path treats model tool objects as proposals and converts accepted analysis into notes. Reminder scheduling uses a separate permission-controlled native bridge. A generic provider registry exists, but the review did not find it wired into application execution. I would not claim broad autonomous multi-tool execution from the presence of that registry.

REMEMBER: Proposal is not execution.

FOLLOW-UP: Distinguish a wired feature, retained code, a stub, and a roadmap item.

FLASHCARDS

Cards 31-33

MEDIUM 31 / NOTEECHOES

Is NoteEchoes fully offline and multilingual?

It implements local inference, embeddings, note storage, and OCR. However, downloads need networking; Apple Speech fallback can use remote recognition; web lookup and feedback are remote-capable. It supports English, Hindi, Telugu, and mixed recognition settings, but the English correction score is not evidence of equal real-audio accuracy across languages.

REMEMBER: Local-first, with explicit remote exceptions.

FOLLOW-UP: Do not claim fully offline operation or measured multilingual parity.

HARD 32 / NOTEECHOES

What would you improve first in NoteEchoes?

I would first add a fresh human-authored holdout and matched stock-versus-adapted evaluation, then measure real-device latency and resource use. I would also fix failed-PDF reimport handling: the current hash deduplication can return an existing failed document without reprocessing it. That is an identified code-grounded issue, not a historical incident I am claiming to have fixed.

REMEMBER: Independent evaluation + device measurement + retry recovery.

FOLLOW-UP: Historical TestFlight evidence does not establish current availability, adoption, or crash-free usage.

EASY 33 / MFLASH

Explain MFlash in one minute.

MFlash is a Python/PyQt macOS assistant with local document retrieval, scoped file operations, and model-assisted proposals. It separates deterministic routes, a structured template classifier, an MPNet ranker, and experimental visual workflows. The execution layer checks authority, state, and results instead of trusting model confidence. Its strongest story is controlled execution and recovery evidence. The source identifies it as a Design Preview.

REMEMBER: Interpret -> propose -> authorize -> execute -> verify.

FOLLOW-UP: Do not describe it as a production-ready general computer-use agent.

Cards 34-36

MEDIUM 34 / MFLASH

Walk through a folder question in MFlash.

The request and attached folder become a typed local_rag_query intent with folder scope. The ordinary scheduler builds a deterministic proposal, checks capability and missing arguments, and constructs an application-policy grant. The retrieval engine reads indexed SQLite chunks, answers narrow labeled fields directly or retrieves evidence for a local answer. Source changes around answering cause rejection rather than publication of stale output.

REMEMBER: Typed intent -> policy -> scoped retrieval -> evidence.

FOLLOW-UP: This ordinary chat route does not ask the structured classifier to choose the action.

MEDIUM 35 / MFLASH

What are the distinct model roles?

The structured classifier is a randomly initialized MLP that predicts templates from request/state features. The MPNet ranker embeds textual state and supplied candidate actions, then scores learned projections. A local content model supports document answers. Visual-model workflows are separate experiments. Keeping these roles distinct makes it easier to evaluate what each component actually contributes.

REMEMBER: Classifier selects templates; ranker orders candidates.

FOLLOW-UP: Neither the structured classifier nor the MPNet ranker is a screenshot encoder.

MEDIUM 36 / MFLASH

Are the 74 actions 74 tools?

No. A historical structured run records 74 learned templates and 803 features. A template includes a tool and an argument-binding pattern, so several templates can relate to the same operation. The count comes from the training data; it is not a universal fixed capability count or proof that 74 end-to-end workflows work.

REMEMBER: 74 templates, not 74 independent tools.

FOLLOW-UP: Do not assume the active checkpoint matches the historical run without checking release identity.

Cards 37-39

MEDIUM 37 / MFLASH

What are observation-bound arguments?

Instead of allowing a model to invent an arbitrary filesystem path, a template binds arguments to trusted observations or restricted derived fields. The workbench checks that the proposed action matches the selected operation and trusted inputs. This sacrifices some flexibility but makes argument provenance inspectable and limits what an incorrect prediction can request.

REMEMBER: Select from observed state; do not invent targets.

FOLLOW-UP: Binding arguments narrows risk but does not remove the need for execution-time validation.

HARD 38 / MFLASH

How is the structured classifier trained?

The implemented trainer uses a random-initialized MLP, cross-entropy, AdamW with default learning rate 0.001 and weight decay 0.01, inverse-frequency sampling, batches up to 64, and gradient clipping at 1. Current checkpoint selection rewards accepted-correct minus accepted-wrong proposals, gated on zero wrong accepted mutations on development. A historical 2,000-step summary used older selection reporting.

REMEMBER: MLP + cross-entropy + cost-sensitive acceptance.

FOLLOW-UP: Do not retroactively apply today's selection rule to historical runs.

HARD 39 / MFLASH

Explain the MPNet ranking architecture.

The backbone is frozen sentence-transformers/all-mpnet-base-v2. Separate state and action projection heads map 768 to 512 to 256 dimensions, with normalized outputs. Training learns similarity between a state and candidate action descriptions. At evaluation, the correct candidate is already supplied. The ranker is not discovering every possible action or verifying its eventual execution.

REMEMBER: Frozen encoder; learned heads; candidate similarity.

FOLLOW-UP: The model can select poorly if the supplied candidate set omits the correct action.

Cards 40-42

HARD 40 / MFLASH

What contrastive loss and negatives did you use?

There are two recipes. One combines 0.7 hard-negative loss with 0.3 symmetric in-batch loss. The other uses per-example gold-versus-negative cross-entropy at temperature 0.07. The provided negatives are authored scenario alternatives or sampled distractors; online hard-negative mining is not established. The evidence does not identify which recipe produced the current root checkpoint.

REMEMBER: Two recipes; do not merge their hyperparameters.

FOLLOW-UP: A negative called hard in code is not proof of measured difficulty.

HARD 41 / MFLASH

What does the ranker's 100% score prove?

A saved report records 1,381 correct selections out of 1,381 development candidate sets. That is candidate ranking, not 100% automation. It excludes candidate discovery, perception, authorization, execution, and end-state verification. The report was not rerun in the supplied inspection. I would use it as narrow development evidence and test fresh complete workflows separately.

REMEMBER: 100% candidate selection != 100% task success.

FOLLOW-UP: Repeated development use and authored negatives limit generalization claims.

HARD 42 / MFLASH

What did the structured evaluation show?

The saved evaluation reports 1,298/1,541 full-argument matches, or 84.23%, with 211 abstentions and zero wrong mutating proposals reported. Abstentions are not successful actions. The report explicitly does not evaluate executor safety. A separate training summary says 82.15%; those results have not been established as equivalent and should not be collapsed.

REMEMBER: 84.23% proposal match; abstention is not success.

FOLLOW-UP: Do not infer a complete confusion matrix from these aggregate counts alone.

Cards 43-45

MEDIUM 43 / MFLASH

How does MFlash authorization work?

The workbench uses trusted caller approval and matching execution grants. Model confidence does not grant authority. Ordinary scheduler operations use application-policy grants, so not every action has a separate explicit confirmation. I would explain the specific path rather than claiming a universal approval rule across the whole application.

REMEMBER: Confidence proposes; trusted policy authorizes.

FOLLOW-UP: The boundary should be audited across every mutation entry point.

HARD 44 / MFLASH

What happens if a file changes after preview?

The workbench captures a source digest during preview. Execution checks arguments against the grant and recomputes the digest. A changed source is rejected. An uncertain pending prior attempt returns needs_review rather than being blindly repeated. These checks reduce stale-state risk, but the briefing explicitly does not establish complete protection against external-process races.

REMEMBER: Preview hash -> grant check -> fresh hash.

FOLLOW-UP: This is a code-grounded scenario, not a claimed production incident.

HARD 45 / MFLASH

How are retries made safer?

The executor uses stable retry keys plus action identity. Completed results are rechecked, changed arguments are rejected, and pending uncertain attempts require review. Reviewed batches use a snapshot digest and verify completed runs before replay. This is checked retry behavior, not a global exactly-once guarantee for every workflow.

REMEMBER: Same key + same action + verified result.

FOLLOW-UP: An idempotency key must be scoped and tied to the intended payload.

FLASHCARDS

Cards 46-48

HARD 46 / MFLASH

How does crash recovery work for file moves?

The safer move path records intent in a SQLite journal, creates a destination hard link, verifies content, removes the source, and records completion. A crash can occur between filesystem and database updates. Recovery inspects actual bytes and recorded state. The journal provides recoverable evidence; SQLite and the filesystem are not one distributed atomic transaction.

REMEMBER: Record -> link -> verify -> unlink -> commit.

FOLLOW-UP: The same-volume path fails rather than silently using a different cross-volume behavior.

MEDIUM 47 / MFLASH

How does undo avoid destroying newer files?

Undo checks the destination's bytes before restoration. If the original path is now occupied, it chooses a collision-safe restored name such as _restored_N rather than overwriting the new occupant. If destination content changed, undo can reject the operation. Restoring something safely requires revalidating current state, not only remembering the original path.

REMEMBER: Verify bytes; preserve occupied paths.

FOLLOW-UP: SHA-256 supports content comparison; it does not provide encryption or authorization.

HARD 48 / MFLASH

What did you actually test in MFlash?

The supplied inspection reports 12 passing isolated executor tests in 0.064 seconds using temporary synthetic files. They cover retries, cancellation, collisions, mismatched arguments, changed sources, and pending receipts. I did not rerun those tests for this guide, and that suite is not evidence of installed-app, model, or production reliability.

REMEMBER: 12 executor tests; narrow, concrete scope.

FOLLOW-UP: Keep historical model reports separate from tests actually executed in that inspection.

FLASHCARDS

Cards 49-51

MEDIUM 49 / MFLASH

Explain the local retrieval implementation.

Documents become 400-character chunks with 80-character overlap. SQLite stores text, metadata, versions, and float32 vectors. Retrieval scans selected vectors with dot products, adds lexical, filename, and identifier bonuses, and sorts results. Different embedding spaces use rank-based fusion. Indexed folders and optional folder scope restrict the search. No separate learned cross-encoder reranker was found.

REMEMBER: 400 / 80; SQLite; semantic + lexical bonuses.

FOLLOW-UP: Simple local scanning has query work that grows with collection size.

HARD 50 / MFLASH

How do you ground answers and handle stale sources?

Generated answers are accepted only when the quoted text appears in supplied source content and the answer appears within that quote. The application returns source evidence. Changes detected around answering produce rag_source_changed. This reduces fabricated spans and stale answers, but does not prove that a quote is relevant, refers to the right entity, or came from correct OCR.

REMEMBER: Source contains quote; quote contains answer.

FOLLOW-UP: A citation is evidence of location, not proof of semantic correctness.

HARD 51 / MFLASH

What happened in the visual-model experiment?

A recorded UI-TARS 7B QLoRA experiment used NF4, rank 16, alpha 32, and dropout 0.05. Results were mixed: original-action matching rose from 25/170 to 28/170, while native Mac grounding fell from 142/226 to 139/226. Synthetic held-out-family matching reached 166/168. Those synthetic gains do not establish reliable native workflows or deployment.

REMEMBER: Synthetic gains; native regression; report both.

FOLLOW-UP: The coordinate matching metric permits tolerance; it is not byte-for-byte exact output.

Cards 52-54

HARD 52 / MFLASH

What are MFlash's most important remaining limits?

The app is not wholly offline: web search, Google Drive, and authentication include remote paths. Complete encryption and multi-user isolation are not established; the journal is a global singleton and retrieval has a shared fallback database path. I would prioritize fresh workflow tests, consistent authorization, privacy review, and fault-injected recovery before making broad production claims.

REMEMBER: Local processing != complete privacy or production readiness.

FOLLOW-UP: These are recommendations; do not present them as completed fixes.

MEDIUM 53 / ENGINEERING

How would you build an enterprise MCP retrieval tool?

I would expose a narrow document-search operation with a validated query, filters, and bounded limit. Identity would come from authenticated context, with permission checks before content reaches the model. Results would include source IDs and version metadata. MFlash's scoped retrieval and NoteEchoes' source-aware indexing offer design lessons, but enterprise tenant authorization is additional work.

REMEMBER: Authenticate -> authorize -> retrieve -> cite.

FOLLOW-UP: This is a proposed design, not a description of either project's MCP implementation.

HARD 54 / ENGINEERING

How would you defend against prompt injection?

I would treat retrieved content and model output as untrusted data. A document cannot grant permission to access a secret or execute a command. I would use narrow tools, argument validation, external policy checks, and scoped execution. MFlash illustrates separating proposals from authority. Literal quote checks alone do not prevent prompt injection.

REMEMBER: Data cannot promote itself into authority.

FOLLOW-UP: No prompt alone guarantees immunity; test malicious documents and tool outputs.

Cards 55-57

MEDIUM 55 / ENGINEERING

How would you apply TDD and domain-driven design?

I would model intent, proposal, authorization, execution, and receipt as separate business concepts, as MFlash's typed boundaries illustrate. Deterministic tests can cover stale inputs, changed arguments, cancellations, and collisions. NoteEchoes adds edit-preservation and tombstone scenarios. Model behavior needs separate evaluation datasets. The briefings show tests and boundaries, not proof that every component was originally developed using TDD.

REMEMBER: Test business rules separately from model quality.

FOLLOW-UP: Dataclasses do not automatically provide comprehensive runtime schema validation.

HARD 56 / ENGINEERING

How would you deploy an AI change safely?

I would bind application version, model identity, dataset hashes, prompts, and evaluation results in an immutable release manifest. I would run contract tests and regression evaluations, then deploy gradually with rollback criteria. The projects contain pieces of that evidence, but neither briefing establishes a complete verified chain from every training run to the current installed artifact.

REMEMBER: Version the system, not only the code.

FOLLOW-UP: NoteEchoes CI includes checks and optional unsigned builds; it does not itself upload to TestFlight.

HARD 57 / ENGINEERING

What would you monitor after deployment?

I would track task success, correction and abstention rates, tool errors, stale-state rejections, latency, resource usage, and cost per successful outcome. For NoteEchoes, I would separate ASR, action interpretation, retrieval, and persistence. For MFlash, I would separate proposal quality from execution outcomes. I would not invent p95 latency or user-volume figures absent measured evidence.

REMEMBER: Measure each stage and the final outcome.

FOLLOW-UP: The projects' diagnostics and receipts are useful starting points, not a full observability platform.

FLASHCARDS

Cards 58-60

HARD 58 / ENGINEERING

How would you optimize cost, latency, and drift?

I would first measure where time and resources go. Both projects use exhaustive local vector search, which can become a bottleneck as collections grow. MFlash worker switching trades reload latency for memory residency. I would benchmark indexing and caching changes, then reevaluate quality. Drift monitoring would use new request/document slices rather than relying indefinitely on familiar synthetic templates.

REMEMBER: Measure -> change one bottleneck -> re-evaluate.

FOLLOW-UP: Do not claim ANN indexing, caches, or production drift detection were implemented unless separately verified.

HARD 59 / ENGINEERING

Design a safe airline rebooking assistant.

I would separate policy retrieval, live reservation reads, option selection, approval, and booking execution. Before a write, I would recheck availability and price and bind approval to the exact proposed change. A timed-out booking would be reconciled by transaction identity before retry. MFlash's stale-state checks and checked retries are useful patterns, but airline business rules belong in authoritative services.

REMEMBER: Search -> propose -> approve -> revalidate -> commit.

FOLLOW-UP: This is a system-design proposal; your resume does not establish airline experience.

HARD 60 / ENGINEERING

How would you evaluate an enterprise RAG assistant?

I would measure retrieval recall and ranking separately from answer correctness, citation support, and abstention. LLM-as-a-judge can scale rubric scoring but needs human calibration and versioning. Authorization requires deterministic checks. Neither project briefing establishes measured retrieval recall or general citation accuracy, so I would propose those evaluations rather than invent results.

REMEMBER: Retrieval quality + answer quality + access correctness.

FOLLOW-UP: Your separate LangGraph codebase project lists Ragas; do not transfer its implementation to these apps.

FLASHCARDS

Cards 61-63

EASY 61 / BEHAVIOR

Explain your Bank of America production experience.

My resume describes OCR-assisted PDF extraction using a fine-tuned model, JSON and field-level validation, scheduled secure API delivery, and a React review interface. I also worked on MCP-based document retrieval and gathered requirements from external clients. The two project briefings provide deeper personal-project implementation detail, but they do not establish the bank's base model, OCR vendor, hosting stack, or exact MCP transport.

REMEMBER: Lead with real production; use projects for deeper mechanics.

FOLLOW-UP: Those bank-specific details remain personal confirmation items.

MEDIUM 62 / BEHAVIOR

How did you measure the 50-60% effort reduction?

My resume describes an estimated reduction in manual effort for the extraction workflow. That is not model accuracy or necessarily measured financial savings. I should explain the actual baseline, remaining review effort, sample, and source of the estimate. The supplied documents do not establish that measurement method, so I cannot honestly fill it in from the project evidence.

REMEMBER: Estimated effort reduction != accuracy.

FOLLOW-UP: Use only the time study or stakeholder estimate you actually used; no fabricated formula inputs.

MEDIUM 63 / BEHAVIOR

Tell me about a difficult design tradeoff.

One concrete tradeoff is NoteEchoes saving a usable note before optional AI enhancement. It protects capture availability, but structured enrichment arrives later and must not overwrite user edits. Another is MFlash's observation-bound templates: they reduce flexibility but make action targets inspectable. I can explain the implemented tradeoffs; personal decision history should match my actual contribution.

REMEMBER: Reliability sometimes means less autonomy.

FOLLOW-UP: Use a real decision you owned when the interviewer asks specifically what you chose.

Cards 64-66

HARD 64 / BEHAVIOR

Tell me about a failure and what you learned.

A useful code-grounded example is NoteEchoes' failed-PDF retry gap: hash deduplication can return an existing failed document without reprocessing it. The lesson is to make deduplication state-aware and distinguish successful completion from mere existence. I would add failed-state retry tests. This is an identified issue and proposed response, not a production incident or completed fix I can claim.

REMEMBER: Exists != completed successfully.

FOLLOW-UP: For a behavioral incident question, use your own actual event, actions, and outcome.

MEDIUM 65 / BEHAVIOR

How do you gather requirements and estimate work?

My resume supports direct client requirements gathering and end-to-end workflow delivery. I translate requests into inputs, required outputs, review conditions, and acceptance criteria. For uncertain model behavior, I would time-box a baseline experiment before committing to an accuracy target. NoteEchoes' distinction between schema and action correctness shows why acceptance criteria must be precise.

REMEMBER: Define success before promising a model score.

FOLLOW-UP: Describe a specific client interaction from your experience if asked for a historical example.

MEDIUM 66 / BEHAVIOR

How would you discuss production readiness honestly?

I would distinguish implemented code, tests actually run, archived model reports, and operational evidence. MFlash is identified as a Design Preview. NoteEchoes has historical TestFlight evidence but no verified current tester count or device reliability metrics in the briefing. My Bank of America resume experience is the stronger production example. Honest scope makes the engineering discussion more credible.

REMEMBER: Code -> tests -> release -> real operations.

FOLLOW-UP: A build script or manifest is not proof of a successful deployment.

Cards 67-69

EASY 67 / BEHAVIOR

What should you say about tools or domains you have not used?

My strongest listed language is Python, my orchestration project uses LangGraph, and my enterprise experience includes APIs and data pipelines. I would describe actual exposure to Java, Semantic Kernel, Copilot Studio, Jenkins, or airline systems without implying equal depth. I can explain transferable architecture and learn the team's stack. Personal projects are not automatically verified open-source contributions.

REMEMBER: Specific strengths; clear boundaries; transferable skills.

FOLLOW-UP: Mention Excel automation directly; give real examples for other Office tools if asked.

EASY 68 / BEHAVIOR

What questions should you ask the interviewer?

I would ask which AI workflows are already in production, what the engineer owns after deployment, how MCP permissions are managed, what evaluation gates block a release, and what success looks like in the first 90 days. I would also ask whether the main challenge is model quality, integration, reliability, latency, or adoption.

REMEMBER: Ownership + evaluation + operations + first 90 days.

FOLLOW-UP: Choose four questions that fit what the interview has not already covered.

EASY 69 / PYTHON

How do you deduplicate IDs while preserving order?

Use a set for membership and a list for ordered results. For each ID, append it only if it has not been seen, then add it to the set. Average time is O(n), and memory is O(n). For an unbounded stream, I would define a deduplication window or persistent store instead of growing an in-memory set forever.

REMEMBER: Set checks; list remembers order.

FOLLOW-UP: Practice empty input, repeated IDs, and all-unique input. See coding drill A.

FLASHCARDS

Cards 70-72

MEDIUM 70 / PYTHON

How do you compute exact-match accuracy correctly?

Align predictions and references by stable example ID, validate that reference fields exist, and define normalization before scoring. Count missing predictions as failures, not as matching missing values. NoteEchoes demonstrates why selected-field agreement and complete-object agreement need different metric names and denominators. An accuracy function is only meaningful when the evaluation contract is explicit.

REMEMBER: Alignment + denominator + equality rules.

FOLLOW-UP: See coding drill B. Never silently zip unequal-length lists.

MEDIUM 71 / PYTHON

How do you bound concurrent API calls?

Use a semaphore to cap active calls, a timeout per attempt, and an explicit error policy. For large inputs, use a bounded worker queue rather than creating all tasks at once. Distinguish an execution timeout from an overall deadline including queue wait. Preserve cancellation and avoid blocking synchronous calls inside the event loop.

REMEMBER: Concurrency limit + timeout + cancellation.

FOLLOW-UP: See coding drill C. A semaphore is not itself a requests-per-second rate limiter.

HARD 72 / PYTHON

How would you design a durable idempotency handler?

Store a scoped key, request hash, state, and result under a uniqueness constraint. Reject a reused key with different arguments. Return verified completed results; report in-progress work; reconcile uncertain outcomes before retrying. MFlash supplies a concrete example of checked receipts and pending attempts requiring review. No local dictionary or database flag alone makes an external side effect exactly-once.

REMEMBER: Same request -> same outcome; uncertainty -> reconcile.

FOLLOW-UP: See coding drill D for a state-machine exercise and failure cases.

QUICK REFERENCE

Numbers worth remembering: NoteEchoes

Model and training

Qwen3-0.6B; local MLX 8-bit inference. Initial configured training: NF4 4-bit + double quantization; rsLoRA rank 32, alpha 64, dropout 0.05; sequence 1,024; learning rate 4e-5; batch 8; accumulation 2; one epoch. Settings are not a verified lineage to every distributed artifact.

Correction data

Train: 6,000 rows = 5,400 actions + 600 normalization. Validation, test, and challenge: 600 each = 540 actions + 60 normalization. Test plus challenge creates the 1,200-row evaluation archive.

Evaluation denominators

Selected fields: 1,040/1,080 = 96.296%. Strict stored flags: 1,097/1,200 = 91.417%. Operational stored flags: 1,200/1,200. Schema stored flags: 1,080/1,080 action rows. Normalization: 120/120. Saved-output rescoring is not fresh inference.

Most important qualification

After identifier normalization, 540/540 test action targets and 540/540 challenge targets matched training templates. The score supports familiar-template regression behavior; independent real-language generalization and improvement over stock Qwen remain unmeasured.

Retrieval and storage

PDF chunks: 1,800 characters / 220 overlap. E5-small: 384 dimensions. Hybrid note candidates: up to 15 lexical + 15 semantic, reciprocal rank contribution 1/(60+rank), default six outputs. Primary note storage: SQLite WAL, synchronous=NORMAL.

QUICK REFERENCE

Numbers worth remembering: MFlash

Structured classifier

Historical run: 74 templates, 803 features, 2,000 steps. Random-initialized MLP; no LoRA. Current trainer defaults include AdamW learning rate 0.001, weight decay 0.01, batches up to 64, gradient clipping at 1. Historical and current selection rules differ.

Ranker

Frozen MPNet; separate state/action projections 768 -> 512 -> 256. Two distinct loss recipes exist. Saved development ranking: 1,381/1,381 = 100%, with the correct candidate supplied. This is not end-to-end automation performance.

Structured proposals and executor tests

Saved full-argument evaluation: 1,298/1,541 = 84.23%, with 211 abstentions and zero wrong mutating proposals reported. Separate older training summary: 82.15%. The supplied inspection ran 12 isolated executor tests; it did not rerun model evaluations.

Retrieval and recovery

Chunks: 400 characters / 80 overlap. Local SQLite vector scan plus lexical bonuses; no learned reranker found. File journal: WAL and synchronous=FULL. That database setting does not make filesystem and database updates jointly atomic.

Visual experiment

Original-action metric: 25/170 -> 28/170. Native grounding: 142/226 -> 139/226 (regression). Synthetic held-out family: 96/168 -> 166/168. The action-coordinate metric permits a tolerance. Report the regression alongside the gains.

Collections and evaluation

A. Deduplicate IDs while preserving order

```
def unique_ids(ids: list[str]) -> list[str]:
    seen: set[str] = set()
    result: list[str] = []
    for item in ids:
        if item not in seen:
            seen.add(item)
            result.append(item)
    return result

assert unique_ids([]) == []
assert unique_ids(["a", "b", "a"]) == ["a", "b"]
```

Explain average O(n) time and O(n) space. For unbounded input, define a window or durable store. Do not claim indefinite in-memory deduplication is scalable.

B. Score one field on aligned examples

```
def field_accuracy(predictions, references, field):
    if len(predictions) != len(references):
        raise ValueError("Unequal lengths")
    if not references:
        raise ValueError("Empty evaluation set")
    if any(field not in r for r in references):
        raise ValueError("Missing reference field")
    correct = sum(
        field in p and p[field] == r[field]
        for p, r in zip(predictions, references)
    )
    return correct / len(references)
```

Contract: lists are already aligned by example ID; every reference has the field. Missing predictions fail. Define normalization explicitly. Test empty data, length mismatch, missing references, null values, and missing predictions.

CODING DRILL C

Concurrency with a clear error policy

Python 3.11+ | Assumes search is an asynchronous callable

```
import asyncio

async def retrieve_many(queries, search, limit=5, timeout=3):
    if limit < 1 or timeout <= 0:
        raise ValueError("Invalid limit or timeout")
    sem = asyncio.Semaphore(limit)

    async def one(query):
        async with sem:
            try:
                async with asyncio.timeout(timeout):
                    result = await search(query)
                return {"query": query, "result": result}
            except TimeoutError:
                return {"query": query, "error": "timeout"}

    return await asyncio.gather(*(one(q) for q in queries))
```

What the code guarantees

At most limit searches run inside the semaphore. Per-call timeout starts after a slot is acquired. Results retain input order. Per-call timeouts become explicit results.

What it does not guarantee

This is not a rate limiter or a total request deadline. Large inputs create many pending tasks. Other exceptions propagate, and gather does not automatically cancel every sibling on the first ordinary failure.

Follow-up changes

Use a bounded queue for a large stream. Decide whether other errors should cancel the batch or become individual results. Preserve cancellation. Retry only suitable transient failures; state-changing operations need an idempotency/reconciliation contract.

DESIGN AND TEST EXERCISE

Coding drill D: durable idempotency

Data model

Record tenant/caller scope, idempotency key, canonical request hash, status, result, and timestamps. Enforce uniqueness on scope + key. Protect transitions against concurrent workers.

Decision rules

New key: durably register work. Existing key with another payload: reject. Completed matching request: return a verified stored result. In progress: report status or wait under a clear contract. Uncertain after a crash: reconcile before a new side effect.

The hardest crash window

The external operation may succeed before the local completion record is committed. A database transaction cannot atomically cover an unrelated filesystem or remote API. Use downstream idempotency where available, inspect durable evidence, and route ambiguous outcomes to review.

Connect to MFlash

Its executor checks stable keys and action identity, rechecks completed results, and blocks pending uncertain attempts. Its file journal records prepared/committed states, but actual filesystem bytes still need reconciliation.

Tests to describe

Two simultaneous requests with the same key; same key and changed arguments; timeout after external success; crash before completion recording; completed receipt whose target changed; cancellation before and during execution; unauthorized retry from another caller.

What the interviewer is checking

Explain the failure model and state transitions before writing code. A process-local dictionary is a useful toy demonstration, not durable distributed idempotency.

RESOLVE THE EARLIER FOLLOW-UPS

What is now answered, and what remains

Answered for NoteEchoes

Exact model/runtime, configured training settings, dataset packages, metric denominator, error slices, template overlap, retrieval/chunking, OCR, graph semantics, capture ordering, edit protection, recovery, and release-evidence limits. See cards 13-32.

Answered for MFlash

Model roles, template count, ranker architecture and loss variants, argument provenance, authorization paths, retries, crash recovery, undo, retrieval, actual isolated-test scope, visual results, and production/privacy limits. See cards 33-52.

Still requires your own account

Which modules and decisions you personally owned; which training runs you executed; which checkpoint matches a current distributed build; actual testers/users; observed incidents and measured latency. Neither repository briefing can establish those personal or operational facts.

Still unresolved at Bank of America

Actual base model, OCR engine, training data size, deployment platform, failed-delivery handling, MCP transport and server/client ownership, and the measurement method behind estimated effort savings. Do not substitute details from your personal projects.

A safe way to answer a missing measurement

"The evidence I can substantiate is [specific implementation or result]. I have not established [missing measurement], so I would not attach a number to it. The next validation would be [concrete test]." For your own history, describe only work and outcomes you actually remember.

REFERENCE NOTES

Sources and evidence boundaries

Primary source 1 | Your resume

Vashisht_Devasani_Av2.pdf, supplied in this chat. Used for Bank of America, Sedin, internship, education, skills, and project-level background. Resume statements are not independently verified operational measurements.

Primary source 2 | MFlash technical briefing

User-supplied "MFlash technical interview briefing", pasted attachment e053a100-1279-4f79-86e6-a753dba2c26c. Sections 2-8 support architecture, training, retrieval, evaluations, and safeguards. Sections 9-12 distinguish scenarios, recommendations, and ownership questions.

Primary source 3 | NoteEchoes technical briefing

User-supplied "NoteEchoes - technical interview briefing", pasted attachment 54db3d86-b5f5-4ca1-9bf1-123ddfb13880. Sections 2-8 support implementation, configured training, metrics, and reliability. Sections 9-12 distinguish limitations and next steps.

Technical references

MCP: https://modelcontextprotocol.io/docs/learn/architecture
Python asyncio: https://docs.python.org/3/library/asyncio-task.html
Transformer: https://arxiv.org/abs/1706.03762
QLoRA: https://arxiv.org/abs/2305.14314
These were consulted in the earlier interview-preparation response.

How to interpret the answers

Implementation statements reflect the supplied reviews, not a fresh repository audit. Archived scores are not rerun inference. Configuration is not proof of executed settings or current distribution. Proposed improvements are not completed accomplishments. Use first-person ownership language only for work you actually did.